

Not Just an Introduction ❌

1. How to Read the Question ??

- *Understand the Problem Statement:* Read the question carefully and ensure you understand the problem. Focus on inputs, outputs, and constraints.
- *Break It Down:* Divide the question into smaller parts. Identify the main goal and the subtasks required to achieve it.
- *Ask "What is Required?":* Think about what the problem demands. Is it a number, a string, or a specific data structure?

2. When to Look at the Approach ??

- *Initial Efforts:* Try solving the problem on your own for at least 20-30 minutes.
- *Stuck? Seek Hints:* If no progress is made, refer to hints or a similar example to get an idea of how to approach the problem.
- *Understand the Logic:* Do not rush to the solution. Focus on understanding the thought process behind the approach.

3. When and Where to Practice Solutions ??

- *Dedicated Platforms:* Use platforms like LeetCode, HackerRank, or Codeforces for structured practice.
- *Practice Daily:* Solve 1-2 problems every day, focusing on different topics like arrays, trees, and dynamic programming.
- *Topic-Wise Practice:* After learning a topic, practice questions specific to

that topic to solidify your understanding.

4. What About Revision Sessions ??

- *First Revision:* Review notes and solved problems after completing a topic.
- *Weekly Revision:* Allocate one day each week to revise all the topics covered so far.
- *Final Revision:* Before exams or interviews, revise everything and solve a mix of easy, medium, and hard problems.

5. Practicing Extra Questions ??

- *Beyond the Basics:* Once comfortable with standard questions, attempt variations and harder problems.
- *Explore New Patterns:* Look for patterns in problems to improve adaptability.
- *Challenge Yourself:* Attempt problems from competitive programming contests for advanced practice.

6. How to Revise ??

- *Handwritten Notes:* Start with your notes. Read through the steps and pseudocode you have written.
- *Solve Again:* Re-solve important and tricky problems without looking at the solution.
- *Focus on Weak Areas:* Identify and revise topics where you face difficulty.

7. Written Practice vs. Coding Practice

??

- *Written Practice: Helps in understanding the flow of logic and improving handwriting speed for exams.*
- *Spend 30% of your time writing pseudocode and explaining logic on paper.*
- *Coding Practice: Builds implementation skills and debugging abilities.*
- *Spend 70% of your time coding problems on a computer.*